

FULLSTACK III — DSY1106

Evaluación Parcial N°2

Implementación de Componentes Frontend y Backend

Plataforma de Libro de Clases Digital

Colegio Bernardo O'Higgins — Coquimbo

INTEGRANTES

Jose Muñoz
Sebastian Santander

Sección: FULLSTACK III_004D

Docente: Prof. Viviana Poblete L.
Mayo 2026

Índice

1. Introducción.....	4
2. Arquitectura Implementada.....	4
2.1 Visión General.....	4
2.2 Infraestructura con Docker Compose.....	4
3. Patrones de Diseño Implementados.....	5
3.1 Patrón MVC (Model-View-Controller).....	5
3.2 Patrón Repository (DAO).....	5
3.3 Patrón Gateway (API Gateway / BFF).....	6
3.4 Patrón Observer (Mensajería Asíncrona con RabbitMQ).....	6
3.5 Patrón Proxy / Feign Client.....	6
3.6 Patrón Component (Frontend React).....	6
4. Arquetipos Maven y Patrones Arquitectónicos.....	7
4.1 Arquetipo Utilizado: spring-boot-starter-parent.....	7
4.2 Patrón Arquitectónico: Microservicios.....	7
4.3 Patrón Arquitectónico: Layered Architecture (N-Tier).....	7
5. Componentes Frontend.....	8
5.1 Stack Tecnológico.....	8
5.2 Estructura del Proyecto Frontend.....	8
5.3 Sistema de Diseño.....	8
5.4 Dashboards por Rol.....	9
5.5 Conexión al Backend — Rol Alumno.....	9
6. Componentes Backend.....	9
6.1 Microservicio Académico (puerto 8081).....	9
6.2 Microservicio Asistencia (puerto 8082).....	10
6.3 Microservicio Comunicaciones (puerto 8083).....	10
6.4 API Gateway (puerto 9090).....	10
7. Pruebas Unitarias y Calidad de Código.....	11
7.1 Tests Implementados.....	11
7.2 Estrategia de Testing.....	11
7.3 Análisis de Calidad con SonarQube.....	12
8. Estrategia de Branching — GitHub Flow.....	12
8.1 Descripción de la Estrategia.....	12
8.2 Estructura de Ramas.....	12
8.3 Flujo de Trabajo.....	12
8.4 Convenciones de Commits.....	12
8.5 Resolución de Conflictos.....	13
9. Repositorios GitHub.....	13
10. Justificación frente a la Rúbrica de Evaluación.....	13
Indicador 1 — Patrones de diseño (10%).....	13
Indicador 2 — Arquetipos y patrones arquitectónicos (10%).....	13
Indicador 3 — Estrategia de branching (5%).....	14
Indicador 4 — Buenas prácticas y pruebas unitarias (5%).....	14
11. Conclusión.....	14

12. Bibliografía.....14

1. Introducción

El presente informe documenta el desarrollo de la Evaluación Parcial N°2 de la asignatura Desarrollo Fullstack III (DSY1106), correspondiente al Colegio Bernardo O'Higgins, institución educacional ubicada en la ciudad de Coquimbo.

En la primera evaluación se diseñó y planificó la arquitectura general del sistema: se definieron los microservicios, se seleccionaron las tecnologías base y se justificaron los patrones arquitectónicos que guiarían la implementación. En esta segunda etapa el equipo avanza hacia la construcción real del sistema, implementando los componentes frontend y backend de acuerdo a los principios de diseño establecidos.

El sistema "Libro de Clases Digital" tiene como propósito reemplazar los procesos manuales y en papel del colegio, centralizando la información académica, la asistencia, las anotaciones y las comunicaciones institucionales en una plataforma digital accesible para cinco perfiles de usuario: Administrador, UTP, Profesor, Apoderado y Alumno.

2. Arquitectura Implementada

2.1 Visión General

La solución sigue una arquitectura de microservicios con un API Gateway como punto de entrada único. Cada microservicio tiene su propia base de datos y se comunica de forma independiente, garantizando el principio de bajo acoplamiento y alta cohesión.

Servicio	Puerto	Tecnología	Responsabilidad
api-gateway	9090	Spring Cloud Gateway	Punto de entrada único, JWT, CORS, enrutamiento
servicio-academico	8081	Spring Boot + MySQL	Estudiantes, Profesores, Asignaturas, Evaluaciones, Notas
servicio-asistencia	8082	Spring Boot + MySQL	Asistencias, Anotaciones + RabbitMQ Publisher + Feign
servicio-comunicaciones	8083	Spring Boot + MongoDB	Mensajes, Notificaciones + RabbitMQ Consumer
frontend	5173	React 18 + Vite + Tailwind	Interfaz de usuario — 5 roles con dashboards propios
sonarqube	9000	SonarQube Community	Análisis de calidad y cobertura de código

2.2 Infraestructura con Docker Compose

Todo el ecosistema está orquestado con Docker Compose, lo que garantiza reproducibilidad y aislamiento de entornos. Se definen healthchecks en cada servicio para asegurar que las dependencias arranquen en el orden correcto:

- db-academico y db-asistencia: instancias MySQL 8.0 con volúmenes persistentes.
- db-comunicaciones: instancia MongoDB 6.0 con volumen persistente.
- rabbitmq: imagen rabbitmq:3-management con consola de administración en el puerto 15672.
- Todos los microservicios esperan a que sus bases de datos pasen el healthcheck antes de iniciar (condition: service_healthy).
- SonarQube: contenedor dedicado para análisis de calidad con persistencia de datos y logs.

```
docker-compose up # Levanta todo el ecosistema
cd frontend && npm run dev # Inicia el frontend en localhost:5173
```

3. Patrones de Diseño Implementados

La evaluación exige justificar al menos 3 patrones de diseño en los componentes frontend y backend. A continuación se documentan los patrones implementados con su justificación técnica.

3.1 Patrón MVC (Model-View-Controller)

Implementado en los tres microservicios del backend. Separa las responsabilidades en tres capas bien definidas:

- **Modelo:** entidades JPA con Lombok (`@Entity`, `@Builder`, `@Data`) que mapean directamente a las tablas de la base de datos. Ejemplo: Estudiante, Nota, Asistencia, Anotación, Mensaje, Notificación.
- **Vista:** en una arquitectura REST, la "vista" está representada por la respuesta JSON serializada por Jackson que cada Controller devuelve al cliente.
- **Controlador:** clases anotadas con `@RestController` y `@RequestMapping` que reciben las peticiones HTTP, delegan la lógica al Service y retornan `ResponseEntity` con el resultado.

Justificación: separa responsabilidades, facilita las pruebas unitarias (se pueden testear Services sin levantar el contexto HTTP) y permite que el equipo trabaje en paralelo sobre las distintas capas.

3.2 Patrón Repository (DAO)

Todos los accesos a la base de datos se abstraen mediante interfaces que extienden `JpaRepository` (MySQL) o `MongoRepository` (MongoDB). Esto desacopla la lógica de negocio del mecanismo de persistencia.

- `EstudianteRepository`, `ProfesorRepository`, `NotaRepository` → `JpaRepository<T, Long>`
- `AsistenciaRepository`, `AnotacionRepository` → `JpaRepository<T, Long>`
- `MensajeRepository`, `NotificacionRepository` → `MongoRepository<T, String>`

Justificación: permite intercambiar el motor de base de datos sin modificar la lógica de negocio. Los métodos CRUD básicos (`findAll`, `findById`, `save`, `delete`) están disponibles sin escribir SQL, y se pueden agregar consultas personalizadas con `@Query` o métodos derivados de nombres.

3.3 Patrón Gateway (API Gateway / BFF)

El api-gateway actúa como Backend For Frontend (BFF): es el único punto de exposición pública del sistema. Implementado con Spring Cloud Gateway, centraliza:

- **Autenticación y autorización:** JwtAuthenticationFilter valida el token JWT en cada petición antes de enrutarla al microservicio destino.
- **Enrutamiento:** las rutas se configuran en application.properties mapeando cada path al servicio correspondiente (/api/estudiantes/** → servicio-academico:8081, /api/ anotaciones/** → servicio-asistencia:8082, etc.).
- **CORS centralizado:** CorsConfig.java configura los orígenes permitidos (localhost:5173) en un único punto, evitando configuraciones duplicadas en cada microservicio.

Justificación: el frontend solo conoce una URL (localhost:9090). La complejidad de la red interna de microservicios queda completamente oculta. Si se agrega un microservicio nuevo, basta agregar una regla al gateway sin tocar el frontend.

3.4 Patrón Observer (Mensajería Asíncrona con RabbitMQ)

Implementado entre servicio-asistencia y servicio-comunicaciones siguiendo el patrón Publicador/Suscriptor:

- **Publicador:** AnotacionEventPublisher publica un AnotacionEvent en RabbitMQ cada vez que se registra una anotación.
- **Suscriptor:** AnotacionEventConsumer en servicio-comunicaciones escucha la cola y genera automáticamente una Notificacion en MongoDB.

Justificación: desacopla temporalmente los servicios. Si el servicio de comunicaciones está caído, RabbitMQ retiene el mensaje en cola y lo entrega cuando el servicio vuelva. El profesor puede seguir registrando anotaciones sin errores, garantizando tolerancia a fallos.

3.5 Patrón Proxy / Feign Client

OpenFeign actúa como proxy transparente entre microservicios para comunicación síncrona. AcademicoFeignClient en servicio-asistencia permite consultar datos de estudiantes al servicio-academico mediante una interfaz declarativa, sin escribir código HTTP manual.

Justificación: simplifica las llamadas inter-servicio, hace el código más legible y permite centralizar la configuración de timeouts y reintentos en un único punto.

3.6 Patrón Component (Frontend React)

El frontend implementa el patrón de Componentes de React, donde cada pieza de la interfaz es una unidad reutilizable y autocontenida. Se distinguen:

- **Componentes de layout reutilizables:** Sidebar y TopBar son componentes definidos en el mismo archivo de cada página, con la misma estructura visual y lógica de navegación. Comparten el mismo contrato de props (onLogout, rut, onMenuClick).
- **Componentes de presentación:** PromedioChip, NotaCell, EstadoBadge, TipoBadge — subcomponentes pequeños que encapsulan la lógica de visualización condicional (color según nota, badge según tipo de anotación).
- **Componente global reutilizable:** NotificacionesPanel.jsx es un dropdown de notificaciones importable en cualquier TopBar con una sola línea: `<NotificacionesPanel />`.

Justificación: la componentización reduce la duplicación de código, facilita el mantenimiento y permite que cada integrante del equipo trabaje en componentes distintos sin conflictos.

4. Arquetipos Maven y Patrones Arquitectónicos

4.1 Arquetipo Utilizado: spring-boot-starter-parent

Todos los microservicios del backend están basados en el arquetipo estándar de Spring Boot (spring-boot-starter-parent versión 3.x con Java 21). Este arquetipo define:

- Gestión centralizada de dependencias y versiones (BOM — Bill of Materials).
- Configuración del compilador Java 21 con soporte a records y text blocks.
- Plugin spring-boot-maven-plugin para generar JARs ejecutables autocontenidos.
- Integración con JaCoCo para generación de reportes de cobertura de tests.

La estructura generada sigue el estándar Maven: src/main/java para código fuente, src/main/resources para configuración y src/test/java para pruebas unitarias.

4.2 Patrón Arquitectónico: Microservicios

El sistema implementa una arquitectura de microservicios donde cada servicio es:

- **Independiente:** tiene su propio pom.xml, su propio Dockerfile y su propia base de datos.
- **Desplegable autónomamente:** el docker-compose levanta o detiene cada servicio sin afectar a los demás.
- **Organizado por dominio:** servicio-academico maneja todo lo relacionado con la información curricular; servicio-asistencia gestiona presencia y anotaciones conductuales; servicio-comunicaciones maneja mensajería y notificaciones.

4.3 Patrón Arquitectónico: Layered Architecture (N-Tier)

Cada microservicio aplica una arquitectura en capas estricta:

Capa	Responsabilidad y clases
Controller	@RestController — recibe peticiones HTTP, delega a Service, retorna ResponseEntity
Service (interface)	Define el contrato de la lógica de negocio — ApoderadoService, NotaService, etc.
ServiceImpl	@Service — implementa la lógica, usa el Repository, lanza excepciones de negocio
Repository	JpaRepository / MongoRepository — acceso a datos sin SQL manual
Model / Entity	@Entity / @Document — mapeo ORM, con Lombok @Builder @Data

Esta separación garantiza que la lógica de negocio (ServiceImpl) se puede probar con JUnit 5 + Mockito sin necesidad de levantar el contexto Spring completo ni una base de datos real.

5. Componentes Frontend

5.1 Stack Tecnológico

El frontend está construido como un proyecto NPM estándar con las siguientes dependencias principales:

Dependencia	Versión	Propósito
react + react-dom	18.x	UI declarativa basada en componentes
vite	5.x	Bundler y servidor de desarrollo rápido
@tailwindcss/vite	4.x	Estilos utilitarios integrados con Vite
react-router-dom	6.x	Enrutamiento SPA con rutas protegidas
zod + zod-validation-error	latest	Validación de esquemas en formularios

5.2 Estructura del Proyecto Frontend

El código fuente sigue una organización modular por responsabilidad:

- **src/context/AuthContext.jsx:** gestión global de sesión (token JWT, rol, RUT) en localStorage.
- **src/services/api.js:** capa de abstracción de todas las llamadas HTTP al backend. Contiene authApi, academicoApi, asistenciaApi y comunicacionesApi con las rutas correctas del gateway.
- **src/components/:** componentes reutilizables: ProtectedRoute.jsx (guarda de rutas por rol) y NotificacionesPanel.jsx (dropdown de notificaciones con badge).
- **src/pages/:** páginas organizadas por rol: Admin/, UTP/, Profesor/, Apoderado/, Alumno/. Cada carpeta contiene el Dashboard y las páginas de detalle del rol.

5.3 Sistema de Diseño

Se definió un sistema de diseño institucional coherente usando las capacidades de tematización de Tailwind v4 (@theme en index.css):

Token	Valor HEX	Uso
--color-navy	#001e40	Color principal institucional, sidebars
--color-navy-mid	#003366	Textos de énfasis, botones primarios
--color-gold	#c9a227	Acento dorado, ítem activo del sidebar
--color-surface	#f4f6fb	Fondo general de la aplicación
Fuente títulos	Playfair Display	Tipografía serif para jerarquía visual
Fuente cuerpo	DM Sans	Tipografía sans-serif legible y moderna

5.4 Dashboards por Rol

Se implementaron dashboards completos para los cinco roles del sistema, cada uno con su sidebar de navegación, topbar con notificaciones, KPIs y páginas de detalle:

Rol	Páginas	Funcionalidades principales
ADMIN	2 páginas	Dashboard con métricas, gestión de usuarios con CRUD local, modales, paginación
UTP	4 páginas	Panel global, supervisión de libros, directorio de usuarios, aprobación de cuentas con maestro-detalle
PROFESOR	5 páginas	Dashboard, mis cursos, detalle de curso, registro de notas, registro de anotaciones
APODERADO	4 páginas	Dashboard, ficha del estudiante, notificaciones con categorías, historial de observaciones
ALUMNO	4 páginas	Dashboard, notas por asignatura, asistencia mensual, anotaciones con filtros — CONECTADO AL BACKEND

5.5 Conexión al Backend — Rol Alumno

Como primera integración real con el backend, se conectaron completamente las 4 páginas del rol Alumno. El flujo de datos implementado es:

- Al entrar a cualquier página del alumno, se recupera el RUT del AuthContext (sessionStorage).
- Se llama a GET `/api/estudiantes/rut/{rut}` para obtener el ID numérico del estudiante en la base de datos.
- Con ese ID se llaman en paralelo (Promise.all) los endpoints `/api/notas/estudiante/{id}` y `/api/asistencias/estudiante/{id}`.
- Los datos reales reemplazan los datos estáticos previos. Mientras carga, se muestran skeleton loaders animados.
- Si ocurre un error (401 expiración, red, etc.) se muestra un banner de error con botón "Reintentar".

Las notas se agrupan por asignatura en el frontend (función `agruparNotasPorAsignatura`) para mostrar el promedio por materia. La asistencia se filtra por mes activo, calculando dinámicamente los contadores de presentes, ausencias y atrasos.

6. Componentes Backend

6.1 Microservicio Académico (puerto 8081)

Gestiona toda la información curricular del colegio. Conectado a MySQL (db_academico).

Entidad	Campos clave	Endpoints REST
Estudiante	nombre, apellido, rut, email, curso	GET/POST/PUT/DELETE /api/estudiantes, GET /rut/{rut}
Profesor	nombre, apellido, rut, especialidad, email	GET/POST/PUT/DELETE /api/profesores
Apoderado	nombre, apellido, rut, email, @ManyToOne Estudiante	GET/POST/PUT/DELETE /api/apoderados, GET /estudiante/{id}
Asignatura	nombre, nivelCurso, @ManyToOne Profesor	GET/POST/PUT/DELETE /api/asignaturas
Evaluacion	nombre, fecha, TipoEvaluacion, @ManyToOne Asignatura	GET/POST/PUT/DELETE /api/evaluaciones
Nota	valor (Double 1.0-7.0), @ManyToOne Estudiante, @ManyToOne Evaluacion	GET/POST/DELETE /api/notas, GET /estudiante/{id}

6.2 Microservicio Asistencia (puerto 8082)

Gestiona la asistencia diaria y las anotaciones conductuales. Conectado a MySQL (db_asistencia). Se comunica con servicio-academico vía OpenFeign.

- **Asistencia:** { estudianteld, asignaturald, fecha, EstadoAsistencia } — IDs directos (sin objetos anidados), lo que simplifica las peticiones del frontend.
- **Anotacion:** { estudianteld, profesorId, descripcion, TipoAnotacion (POSITIVA/NEGATIVA/NEUTRAL), fecha } — al guardarse, AnotacionEventPublisher publica un evento en RabbitMQ.
- **Feign:** AcademicoFeignClient permite validar que el estudianteld existe en el servicio académico antes de registrar la asistencia.

6.3 Microservicio Comunicaciones (puerto 8083)

Gestiona mensajería y notificaciones usando MongoDB como base de datos de documentos. Consume eventos de RabbitMQ.

- **Mensaje:** { remitenteld, destinatariold, asunto, contenido, fechaEnvio, leído } — MongoDB Document con id String.
- **Notificacion:** { destinatariold, titulo, mensaje, fecha, leida, TipoNotificacion } — creada automáticamente por AnotacionEventConsumer al recibir eventos del bus.

El flujo completo funciona así: el profesor registra una anotación negativa → servicio-asistencia la persiste en MySQL → publica AnotacionEvent en RabbitMQ → servicio-comunicaciones consume el evento → crea una Notificacion en MongoDB → el apoderado la ve en su panel de notificaciones.

6.4 API Gateway (puerto 9090)

Implementado con Spring Cloud Gateway + Spring Security. Centraliza:

- **Autenticación:** POST /auth/login devuelve { token, role, rut }. BCryptPasswordEncoder strength 10.
- **Usuarios de prueba:** definidos en AuthService con hashes BCrypt. 5 roles: ADMIN, UTP, PROFESOR, APODERADO, ALUMNO.
- **Enrutamiento:** application.properties define las reglas: /api/estudiantes/** → :8081, /api/asistencias/** → :8082, /api/mensajes/** → :8083.
- **CORS:** CorsConfig.java permite peticiones desde localhost:5173 y expone el header Authorization.

7. Pruebas Unitarias y Calidad de Código

7.1 Tests Implementados

Se implementaron pruebas unitarias para todos los métodos de la capa de servicio (ServiceImpl) usando JUnit 5, Mockito y AssertJ. Las pruebas cubren los tres microservicios:

Microservicio	Clases de test	Métodos cubiertos
servicio-academico	ApoderadoServiceImplTest, AsignaturaServiceImplTest, EstudianteServiceImplTest, EvaluacionServiceImplTest, NotaServiceImplTest, ProfesorServiceImplTest	listarTodos, buscarPorId, buscarPorRut, guardar, actualizar, eliminar
servicio-asistencia	AnotacionServiceImplTest, AsistenciaServiceImplTest	listarTodos, buscarPorId, buscarPorEstudiante, guardar, actualizar, eliminar
servicio-comunicaciones	MensajeServiceImplTest, NotificacionServiceImplTest	listarTodos, buscarPorId, buscarPorDestinatario, guardar, eliminar

7.2 Estrategia de Testing

Cada clase de test sigue el patrón Arrange-Act-Assert (AAA) con las siguientes convenciones:

- @ExtendWith(MockitoExtension.class) — sin contexto Spring, tests puros y rápidos.
- @Mock en los repositorios — Mockito simula el acceso a la base de datos.
- @InjectMocks en el ServiceImpl — inyección del mock al servicio bajo prueba.
- when(repo.findById(1L)).thenReturn(Optional.of(entidad)) — configuración del comportamiento esperado.
- assertThat(resultado).isNotNull() con AssertJ para aserciones legibles.

Este enfoque garantiza que la lógica de negocio se prueba de forma aislada, sin dependencias de infraestructura, lo que hace los tests deterministas y ejecutables en cualquier entorno CI/CD.

7.3 Análisis de Calidad con SonarQube

Se integró SonarQube Community como contenedor Docker en el docker-compose. Cada microservicio incluye un archivo sonar-project.properties con la configuración para ejecutar el análisis:

```
mvn clean verify sonar:sonar -Dsonar.host.url=http://localhost:9000
```

JaCoCo genera los reportes de cobertura en formato XML que SonarQube consume para mostrar el porcentaje de líneas y ramas cubiertas por los tests.

8. Estrategia de Branching — GitHub Flow

8.1 Descripción de la Estrategia

El equipo adoptó GitHub Flow, una estrategia de branching liviana y adecuada para equipos pequeños con entregas continuas. El principio fundamental es que la rama main siempre contiene código funcional y desplegable.

8.2 Estructura de Ramas

Rama	Propósito
main	Rama principal protegida. Código estable, revisado y funcional. Solo recibe merges mediante Pull Request.
feature/backend-microservicios-[nombre]	Desarrollo de cada microservicio backend. Rama propia por integrante.
feature/frontend-dashboard-[rol]-[nombre]	Desarrollo de cada dashboard del frontend. Rama propia por rol y por integrante.
feature/api-conexion-[nombre]	Integración del frontend con el backend (api.js, AuthContext, conexión Alumno).
hotfix/[descripcion]	Correcciones urgentes directamente sobre main si es necesario.

8.3 Flujo de Trabajo

Cada integrante del equipo siguió el siguiente proceso para contribuir al proyecto:

1. Crear rama desde main: `git checkout -b feature/nombre-funcionalidad`
2. Desarrollar la funcionalidad con commits descriptivos y atómicos.
3. Push a GitHub y abrir Pull Request hacia main con descripción del cambio.
4. Revisión del compañero: code review, prueba local y aprobación.
5. Merge a main (squash o merge commit según el tamaño del cambio).
6. Eliminar la rama de feature después del merge para mantener el repositorio limpio.

8.4 Convenciones de Commits

Se utilizó la convención Conventional Commits para mantener un historial legible:

```
feat(alumno): conectar AlumnoDashboard al backend con datos reales
```

```
fix(gateway): agregar /api/apoderados/** al enrutamiento
test(academico): agregar tests unitarios para NotaServiceImpl
docs(readme): actualizar instrucciones de ejecución con docker-compose
```

8.5 Resolución de Conflictos

Los conflictos más comunes surgieron en archivos compartidos como App.jsx (cuando ambos integrantes agregaron rutas simultáneamente) y api.js. Se resolvieron mediante:

- Comunicación previa antes de modificar archivos compartidos (coordinación por WhatsApp y reuniones de sincronización).
- git rebase en lugar de merge para mantener un historial lineal cuando fue posible.
- Revisión conjunta del diff en GitHub antes de aprobar el Pull Request.
- En conflictos de App.jsx: el segundo integrante en completar el PR actualizaba su rama con git pull origin main --rebase antes de hacer push.

9. Repositorios GitHub

El proyecto está versionado en repositorios separados para frontend y backend, siguiendo la convención de nombres indicada por la docente.

Componente	Repositorio	Contenido
Backend (monorepo)	colegio-ohiggins-backend-munoz-santander	api-gateway + 3 microservicios + docker-compose + SonarQube
Frontend	colegio-ohiggins-frontend-munoz-santander	React 18 + Vite + Tailwind v4, 5 roles, 19 páginas

Ambos repositorios tienen al colaborador profevivasol agregado como indicó la docente. Cada rama de feature evidencia el trabajo individual de cada integrante.

10. Justificación frente a la Rúbrica de Evaluación

Indicador 1 — Patrones de diseño (10%)

Se implementaron 6 patrones de diseño documentados con código real: MVC, Repository, Gateway/BFF, Observer (RabbitMQ), Proxy (Feign), Component (React). Cada uno está justificado según el problema que resuelve en el contexto del proyecto.

Indicador 2 — Arquetipos y patrones arquitectónicos (10%)

Se utilizó el arquetipo spring-boot-starter-parent para los tres microservicios, garantizando coherencia en la estructura del proyecto. Se aplicaron los patrones arquitectónicos Microservicios y Layered Architecture (N-Tier) con evidencia en el código fuente.

Indicador 3 — Estrategia de branching (5%)

Se implementó GitHub Flow con ramas por funcionalidad y por integrante, Pull Requests hacia main, y convenciones de commits documentadas. Se describen ejemplos reales de resolución de conflictos.

Indicador 4 — Buenas prácticas y pruebas unitarias (5%)

Tests unitarios en los 3 microservicios (10 clases de test) usando JUnit 5 + Mockito + AssertJ. Análisis de calidad con SonarQube + JaCoCo integrado en el docker-compose. Código organizado en capas con separación clara de responsabilidades.

11. Conclusión

En esta segunda evaluación el equipo logró implementar de manera concreta la arquitectura diseñada en la evaluación anterior, pasando del plano teórico a un sistema funcional y ejecutable.

Los tres microservicios del backend están operativos con sus bases de datos, tests unitarios y análisis de calidad. El frontend cuenta con 19 páginas distribuidas en 5 roles, todas conectadas al API Gateway mediante JWT. Como primer hito de integración real, las 4 páginas del rol Alumno consumen datos reales del backend, demostrando que la arquitectura end-to-end funciona correctamente.

La arquitectura de microservicios, lejos de ser una complejidad innecesaria, demostró sus ventajas: cuando el sistema de comunicaciones está caído, el módulo académico sigue funcionando; cuando se necesita escalar el registro de asistencia, solo se escala ese contenedor. El uso de RabbitMQ garantiza que ninguna notificación se pierda aunque el consumidor esté temporalmente caído.

Como equipo concluimos que la combinación de patrones bien fundamentados, una infraestructura dockerizada y una estrategia de branching ordenada son los pilares que hacen posible el desarrollo colaborativo y de calidad en proyectos de esta escala.

12. Bibliografía

- Spring Boot. (s.f.). Spring Boot Reference Documentation. <https://spring.io/projects/spring-boot>
- Spring Cloud Gateway. (s.f.). Spring Cloud Gateway Reference. <https://spring.io/projects/spring-cloud-gateway>
- OpenFeign. (s.f.). Spring Cloud OpenFeign. <https://spring.io/projects/spring-cloud-openfeign>
- RabbitMQ. (s.f.). RabbitMQ Documentation. <https://www.rabbitmq.com/documentation.html>
- Docker Inc. (s.f.). Docker Documentation. <https://docs.docker.com/>
- React. (s.f.). React Documentation. <https://react.dev/>
- Vite. (s.f.). Vite Guide. <https://vitejs.dev/guide/>
- Tailwind CSS. (s.f.). Tailwind CSS v4 Documentation. <https://tailwindcss.com/docs/>

- SonarQube. (s.f.). SonarQube Documentation.
<https://docs.sonarsource.com/sonarqube/>
- Fowler, M. (2002). Patterns of Enterprise Application Architecture.
Addison-Wesley.
- Duoc UC. (2026). Apuntes de clase: Desarrollo Fullstack III. Asignatura DSY1106.